

SWS3009B – Tele-Robotic

Sensors / Actuators

Outline

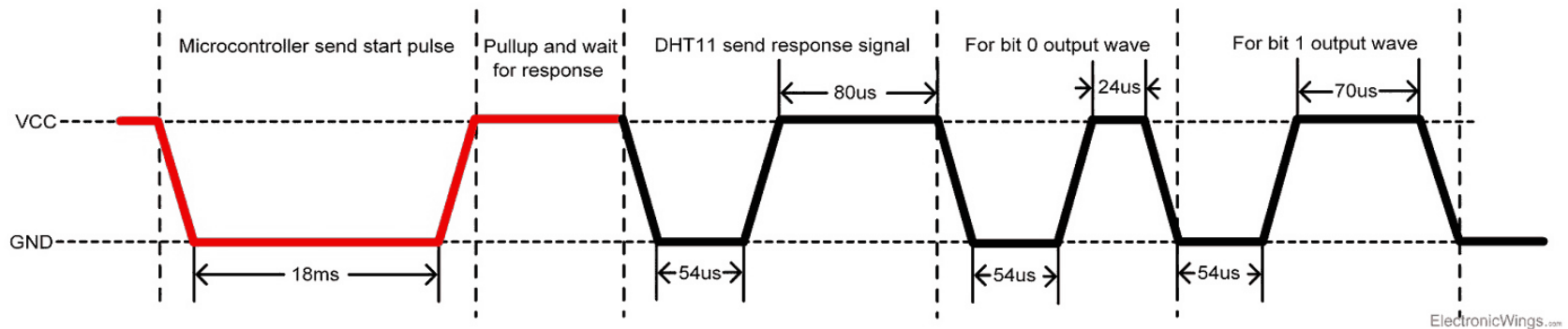
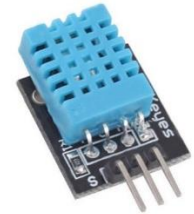
- Accelerometer / Gyroscope / Compass
- Wheel Encoders
- PID Control

Foreword

- All components in this set of notes are **optional**
 - i.e. NOT needed for basic model
- Included here for your own learning and maybe useful for advanced model
- You are advised not to include them first if you are new to Arduino 😊
 - If you want to try, keep the code separate and don't mix with your basic model's code

1-wire Temp Sensor

Timing is everything!



Sends back a 40bit message with the temperature/humidity information, and then goes back to sleep

More than 1 to 1?

Serial communication:

- ❑ Simple but limited to 1-to-1 communication.

What if we want to:

- ❑ 1 sensor shared by two or more microcontroller?
- ❑ 1 microcontroller connecting to many sensors but can't use more than 2 wires?

Inter-Integrated Circuit or **I2C** (also known as **Two Wired Interface TWI**):

It is a synchronous, serial communication protocol, that supports multiple masters/slaves.

It also is a **bus** protocol and allow more flexible connection patterns

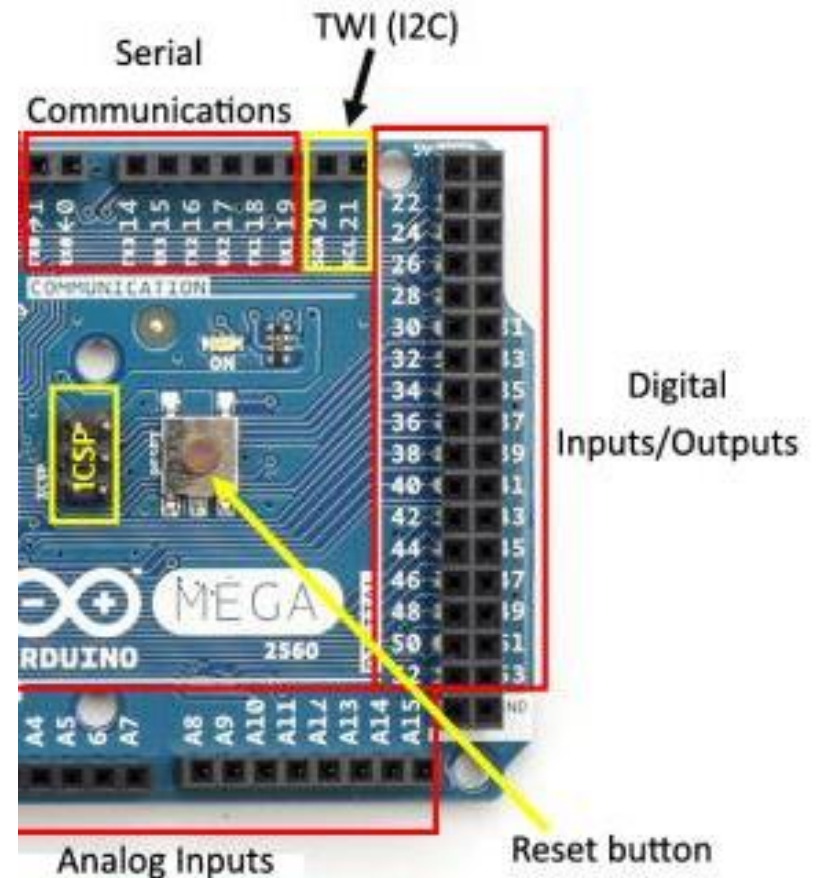
I2C: The two wires

I2C protocol uses two wires:

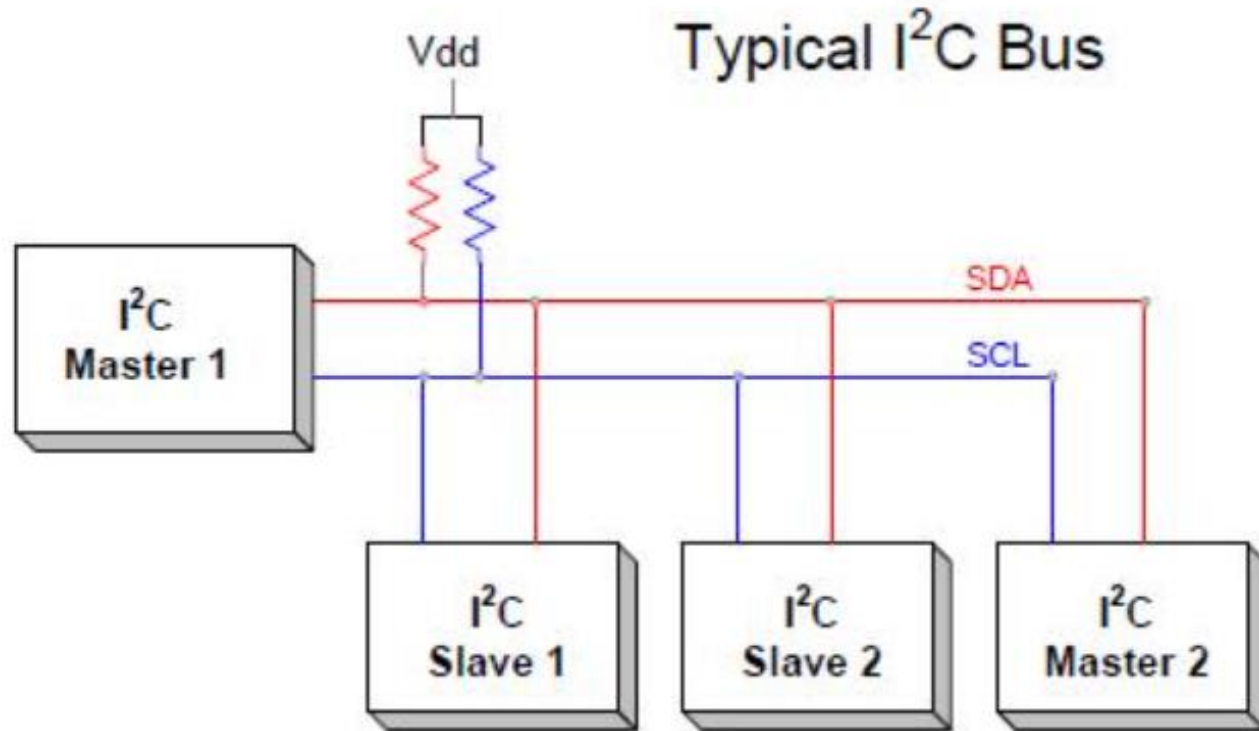
- ❑ Serial Clock (SCL)
- ❑ Serial Data (SDA)

Key idea:

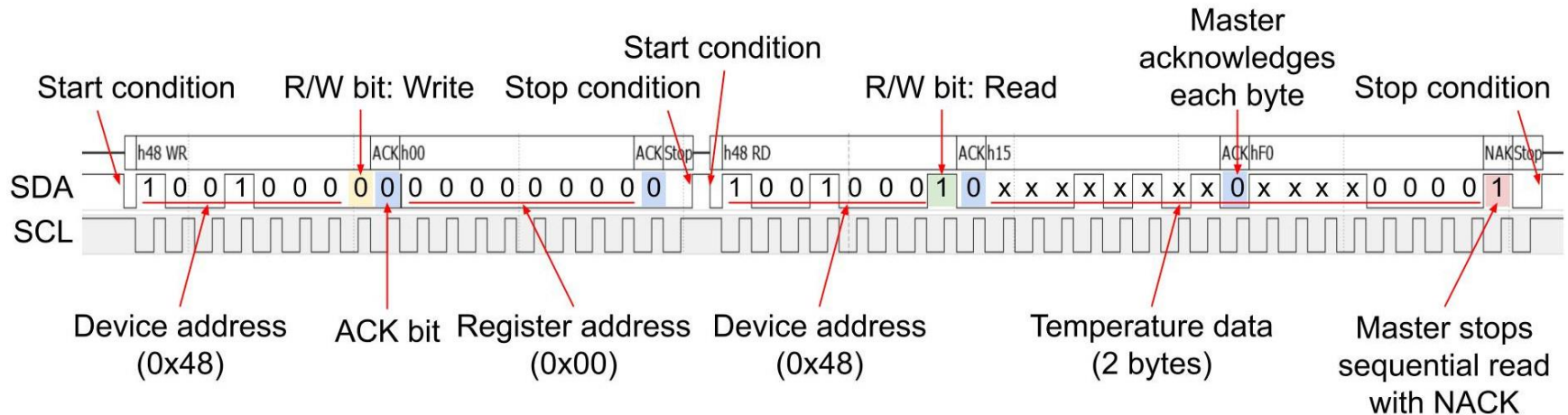
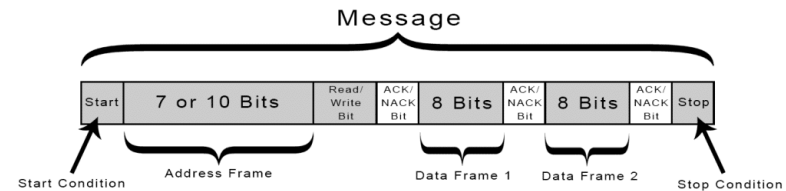
- The Serial Clock establish the same clock among connected devices
- Bits will be sent / read according to this global clock



I2C: Example Master-Slave Connection



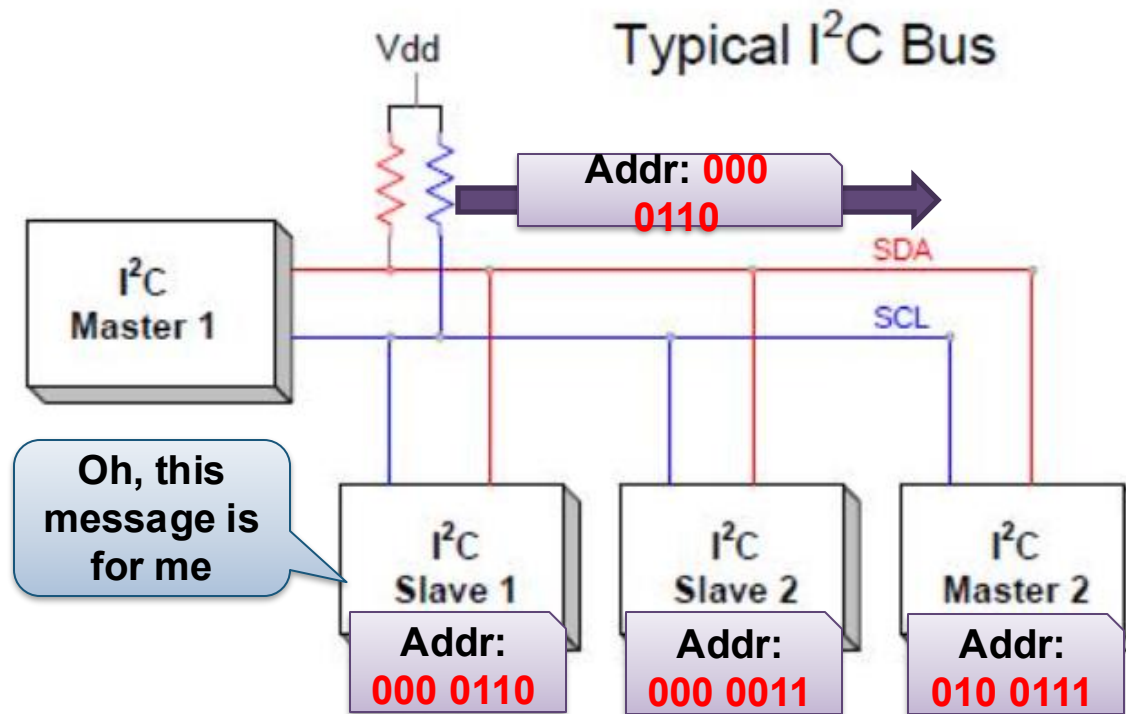
I2C: Message Format



Key Point:

Address can be used to identify a connected device. Tend to be 7 bits (there is a 10-bit extension).

I2C: Message & Address



INERTIAL MEASUREMENT UNITS

Measuring Movement

So how do we measure movement?

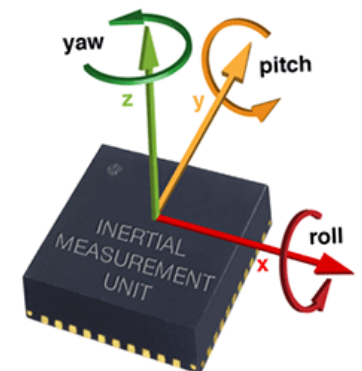
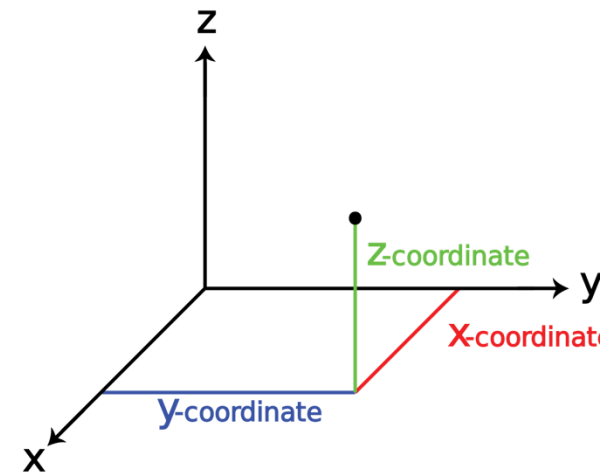
A starting point and an ending point?

Displacement? Velocity?

Acceleration?

And with respect to what? Position?

What about rotation? Deformation?



Inertial Measurement Units (IMU)

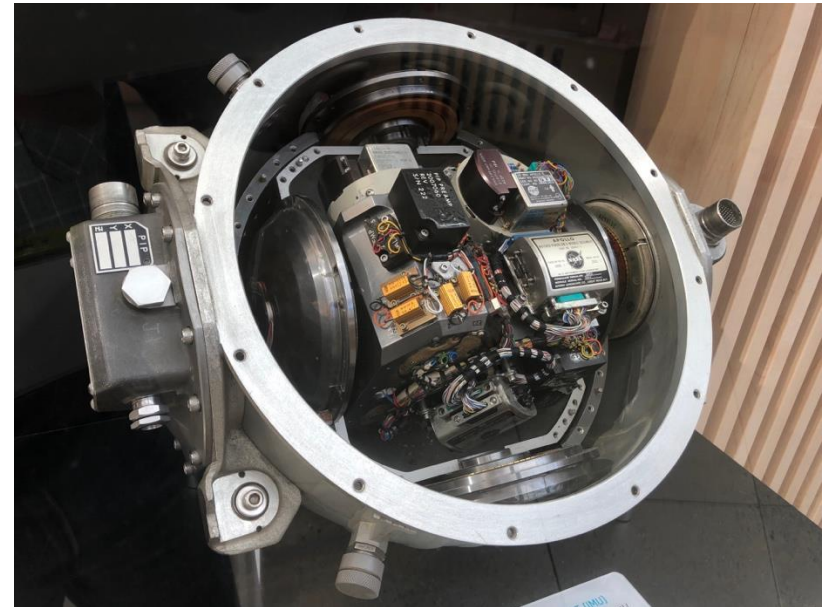
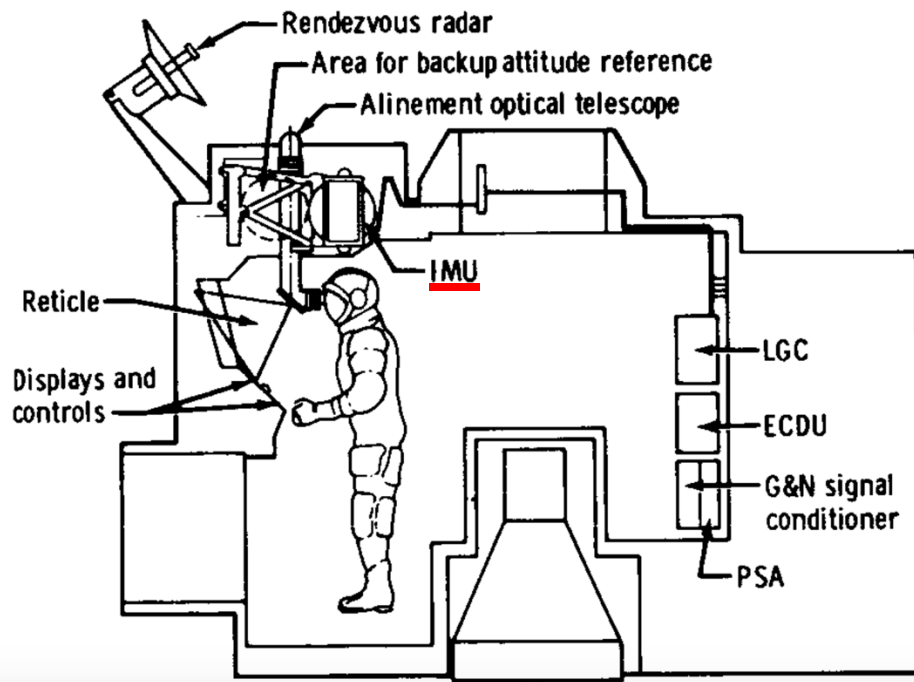
IMUs are physical/electrical systems for measuring acceleration, rotation rate, and orientation. It was first used for navigation in planes in the 1930s.

We measure: **acceleration** using **accelerometers**
rotation rate using **gyroscopes**
orientation using **magnetometers**
(on earth anyway)

An exciting real world test of the technology was Charles "Doc" Draper and his research team's work for the **Apollo (moon landing) space programme** in the 1960s.



Apollo Inertial Measurement Unit



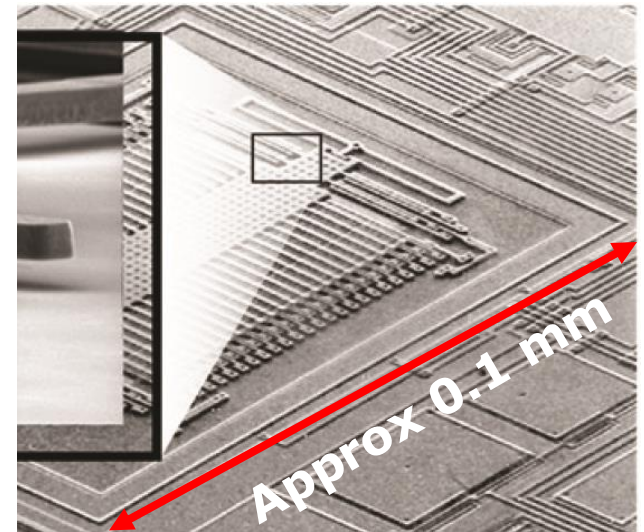
←→
Approx 36 cm

Microelectromechanical Systems

Modern IMUs are much smaller (just a few millimetres across).

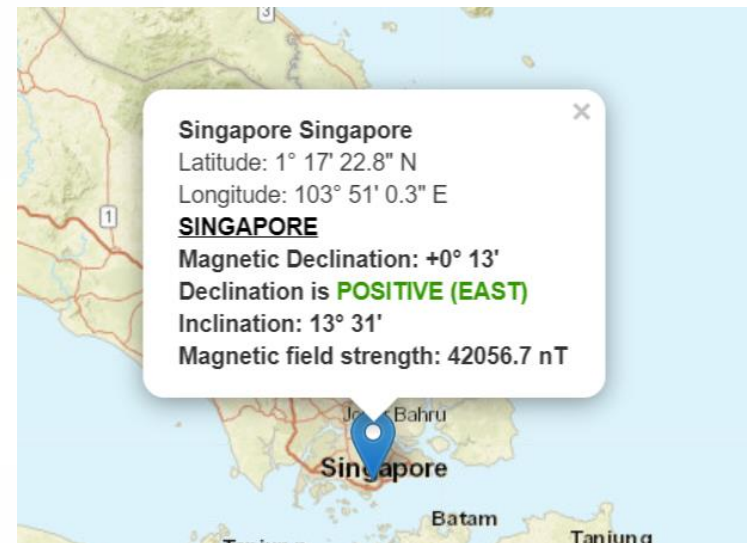
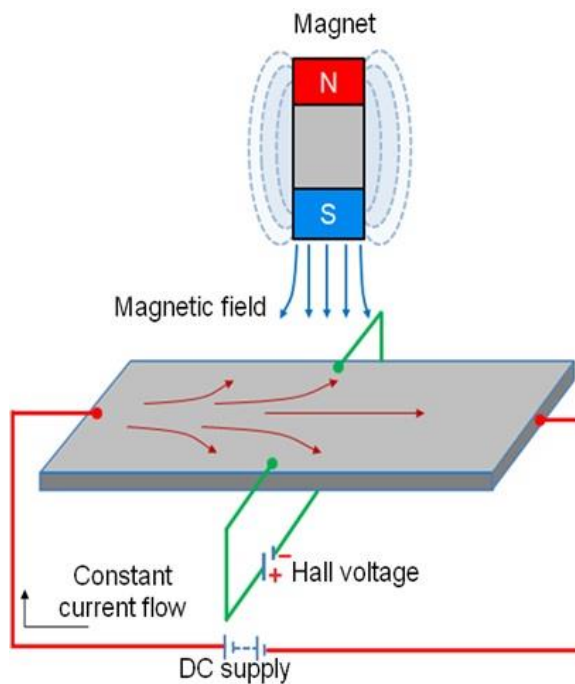
You have a whole bunch
Of them on you right now.
In your phone, laptop,
and any wearables.

But like the Apollo IMU,
they are still physical systems!



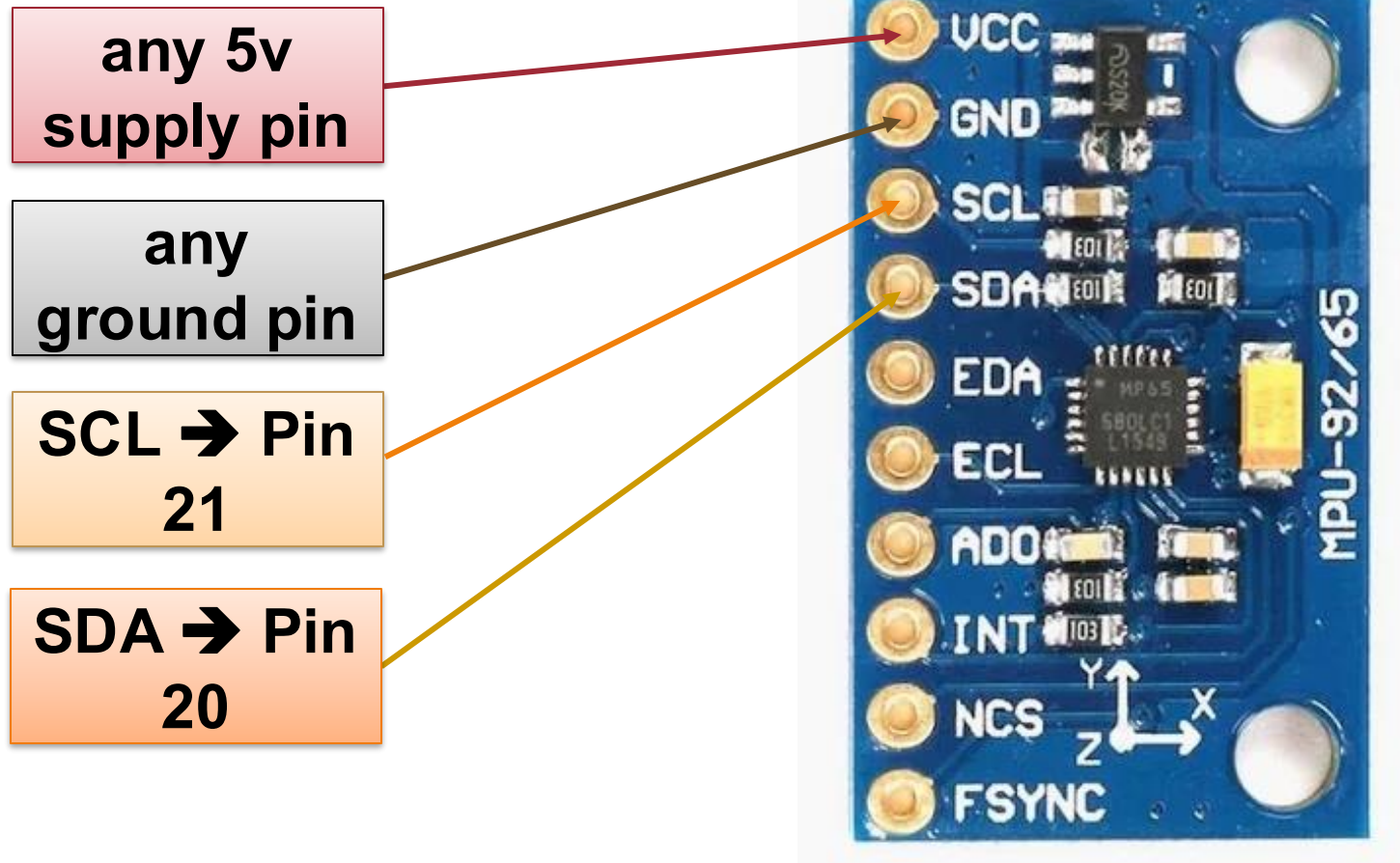
Compass: Magnetometer

- Basic Idea:
 - ❑ Measure the magnetic flux, e.g. **Hall Effect**
- Need to adjust for declination for most sensors



We are using the MPU6500

- It is an accelerometer and gyroscope and uses I2C!



Let's test out the I2C connection

- Investigate the technical specification of the MPU6500.
- What range of acceleration and rotation rate can it measure?
- What is the default I2C address (AD0 is low)?
- Let's test, run I2CScanner.ino. What is returned?

Adding Library & Example Code

- Install the library MPU9250_WE
- Open Examples > MPU9250_WE > MPU6500_all_data
- Read through the program and then run it. What do you observe? Try moving the IMU.

Adding Library & Example Code

- What are the units?
- To get accurate measurements you may need to investigate!

Another option is the use of an attitude and heading reference system (AHRS). However, this is far more complex!

WHEEL ENCODERS

Wheel Travel Distance

■ Basic Idea:

- If we know the number of wheel rotations, we can deduce the **distance travelled**

➔ We need to be able to tell when a wheel make one full rotation (one revolution)....

■ Many **speed encoders**:

1. Use **magnetic** sensors
2. Use **light** sensors ...



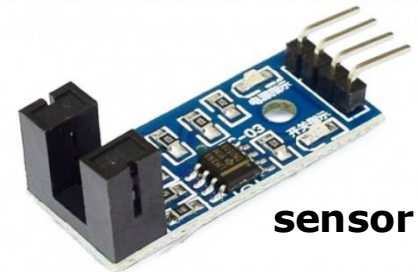
8-pole
magnet



sensor



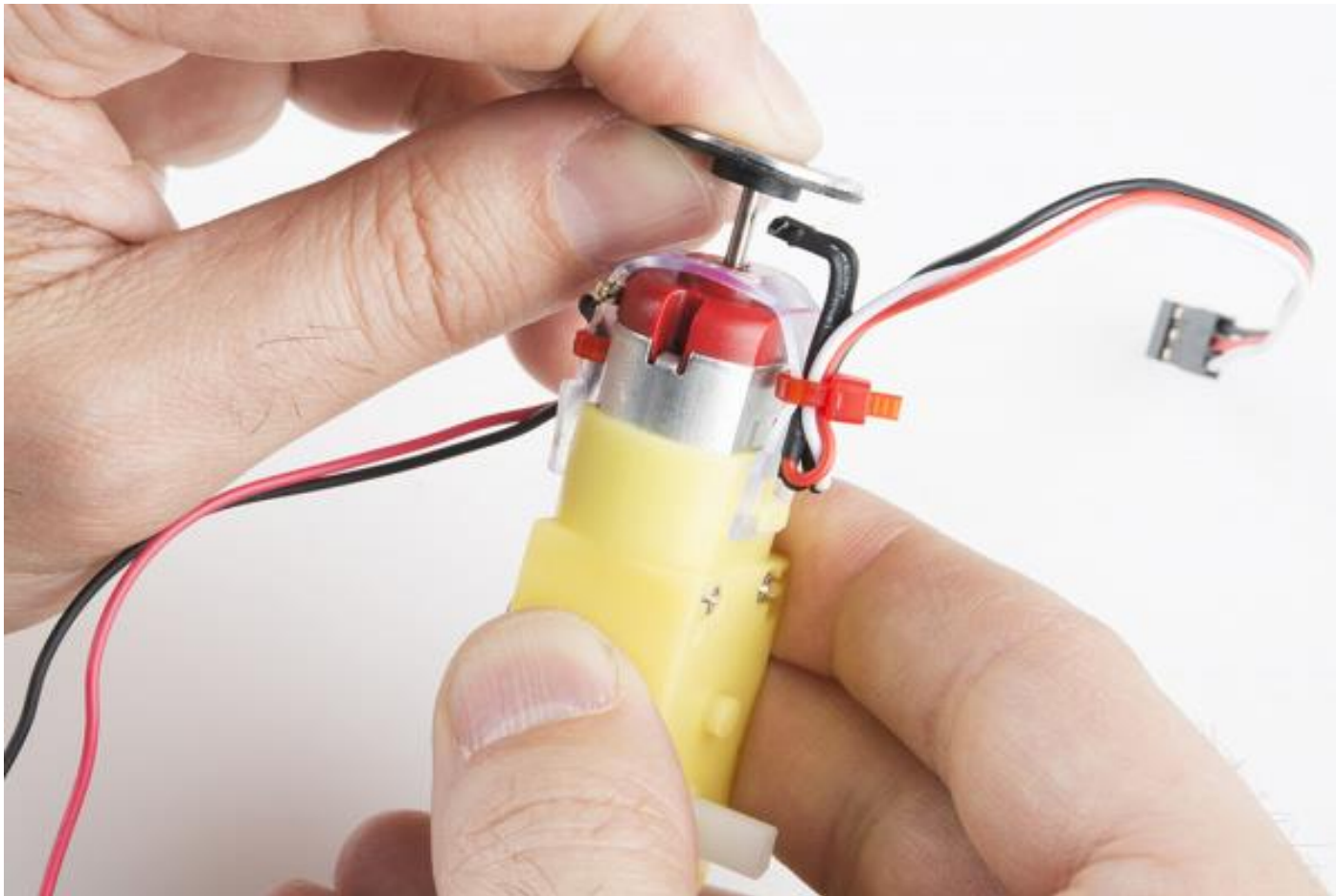
code
disc



sensor

Magnetic sensors

Consists of two neodymium 8-pole magnets with rubber hubs and two hall-effect sensors



Wheel Travel Distance: Basic Test

- Let's use the **Tachometer_Basic.ino** program to get a basic understanding of the encoder
- Plug in one of the sensor wires to Arduino:
 - Red wire → any 5v Pin
 - Black wire → any ground pin
 - White wire → **pin 2**
- Turn the associated wheel and observe the change of values

Wheel Travel Distance: Basic Test

- You should notice that the number of changes is far too many as compared to the turns you made!
 - ❑ Unfortunately, the encoder output is actually not "squarish" (i.e. no $0 \rightarrow 1$ or $1 \rightarrow 0$ cleanly)
 - ❑ i.e. there are fluctuation in the signal ☹
- Need to do "de-bouncing"
 - ❑ A technique that ignore the input value for a short period of time to cancel out the fluctuation.

Wheel Travel Distance: Basic Test

- Use the **Tachometer_Complete.ino** program and observe the value now
 - You may need to change the "threshold" for your encoder
- Can you explain the code?

Move with “Precision”!

- Change the MotorShield program such that:
 - ❑ Forward and backward now takes two numbers **S** and **D**
 - ❑ **S = speed setting**
(1 = slow, 2 = medium, 3 = max)
 - ❑ **D = distance to be travelled in cm**
(Precision within 5cm is acceptable)
- Example:
 - ❑ '**w 2 10**' = move forward using medium speed for 10cm
 - ❑ '**s 3 7**' = move backward full speed for 7 cm

PROPORTIONAL INTEGRAL DERIVATIVE CONTROLLER

Disclaimer

- Just like RTOS, control system is an **optional** feature you can consider
 - It is NOT compulsory!
- Control system taught in this section is very useful, but **takes quite some time** to get it right in practice
 - Be aware of the time commitment before making a decision 😊

The Problem: Controlling Hot Air Balloon

- How would you control the burner so that the balloon reach a certain height?

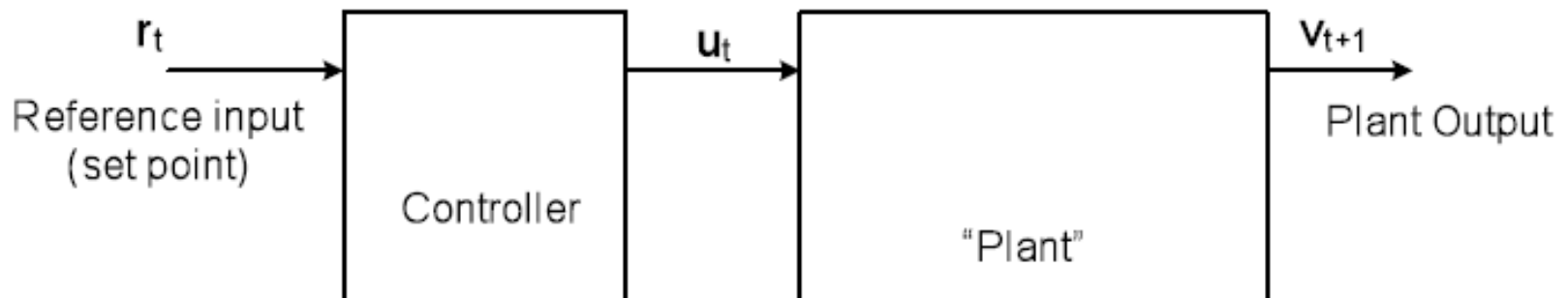


The Problem: Controlling Hot Air Balloon

- Suppose the control knob is at "3" now, and the hot air balloon is stabilizing at 800m
 - What would you do if we want to rise up to 1,200m?
- Notice that the **control knob value does not directly indicate the height** of the balloon
 - i.e. there is a transfer function between the knob value at the actual lift generated

Control System: Definitions

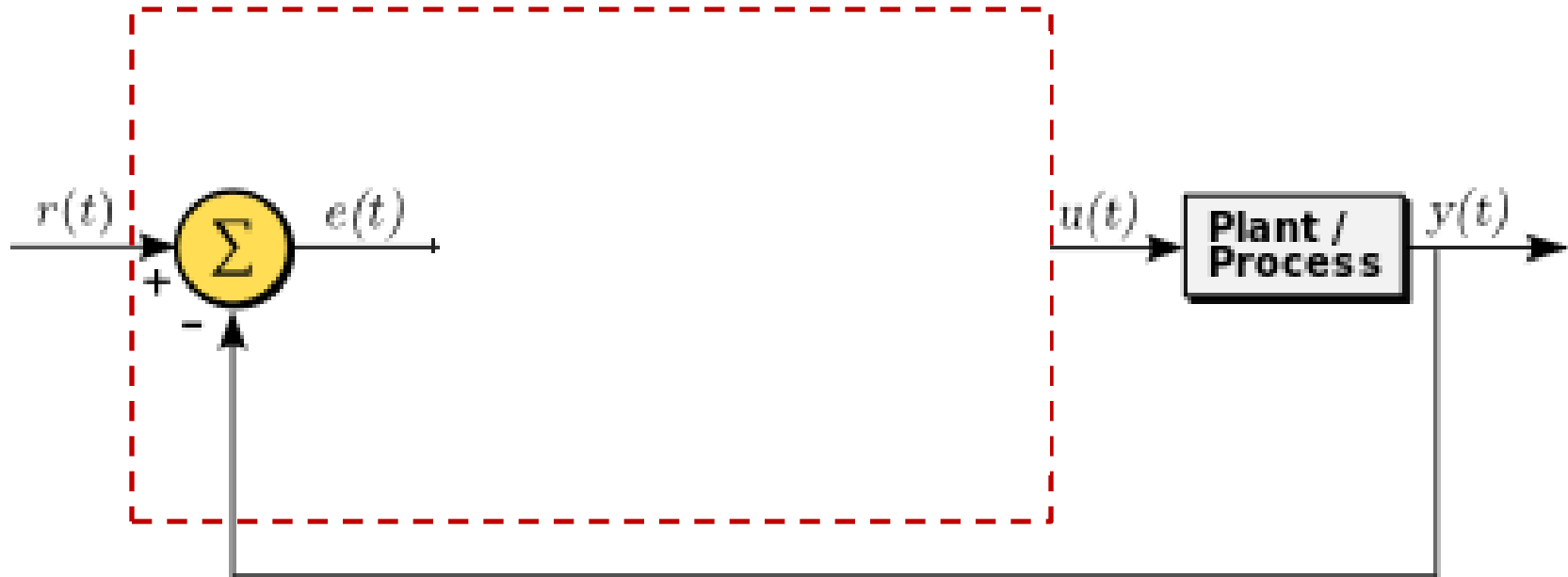
- **Plant** = System / Process we want to control
- **Set point** = Output value we want the system to achieve
- **Control input** = Some values we put into the system to change its behavior



Control System: **How?**

- An intuitive idea is to use the **error** as a way to change the control input
 - **Error** = **Set Point** – **Current Value**
- What kind of **features** / **characteristics** of the **error** can we use?
 - How big is the error?
 - How much has the error accumulated?
 - How fast is the error changing?

Introducing **PID** Controller

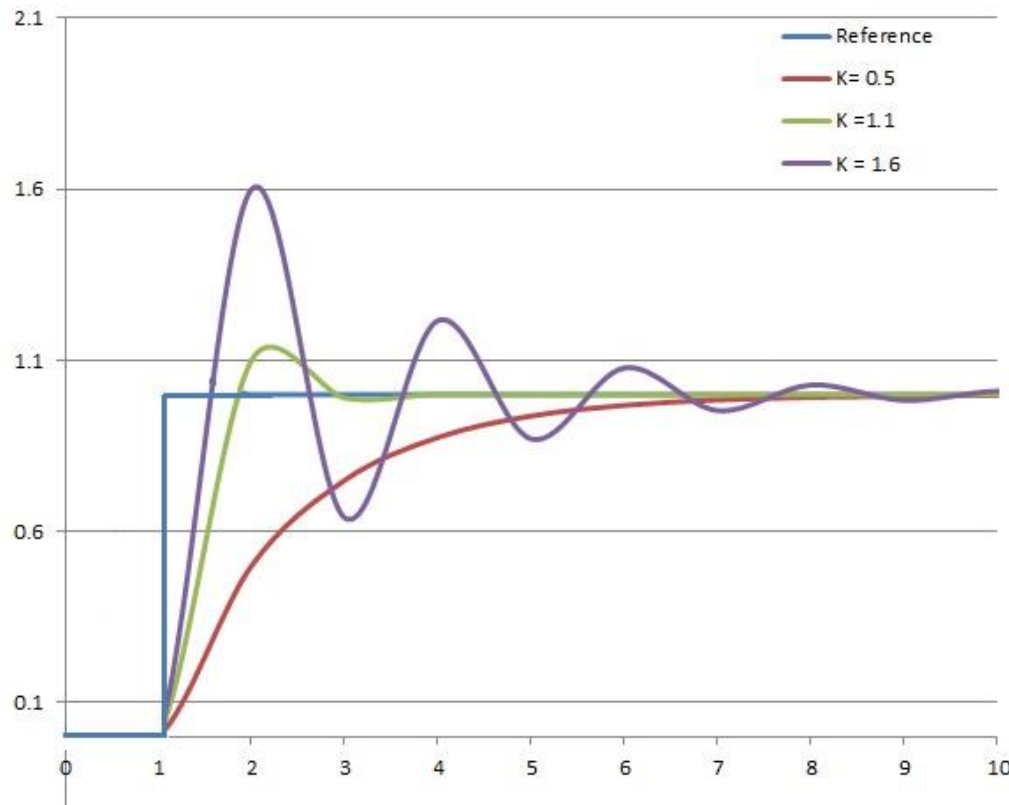


$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Proportional Term

$$K_p e(t)$$

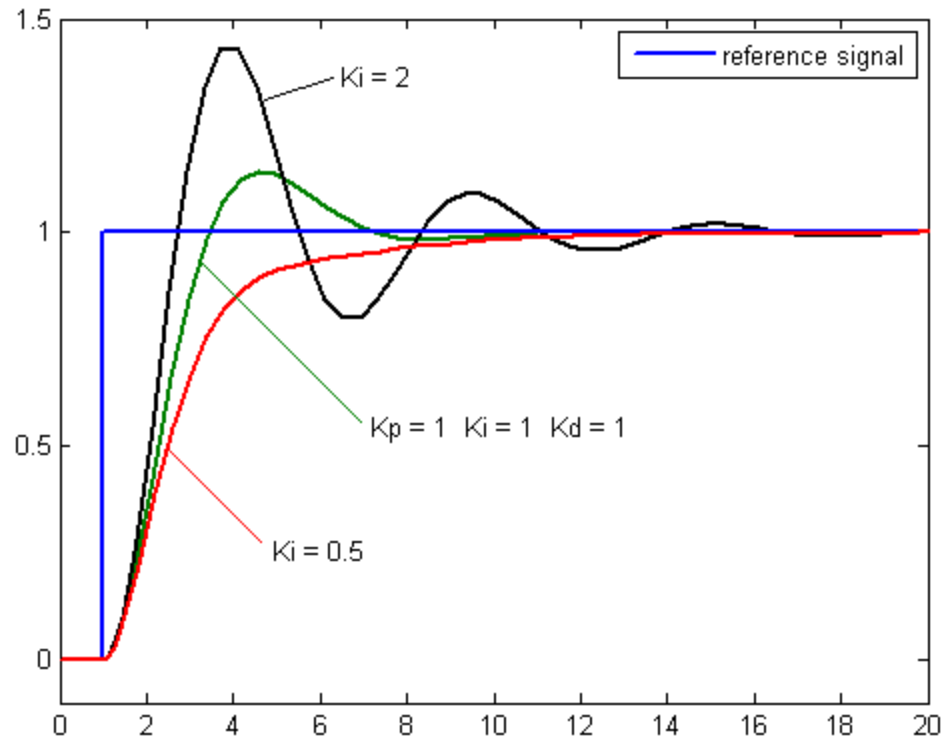
- P accounts for **present values** of the error
 - E.g. if the error is large and positive → the control output will also be large and positive



Integral Term

$$K_i \int_0^t e(\tau) d\tau$$

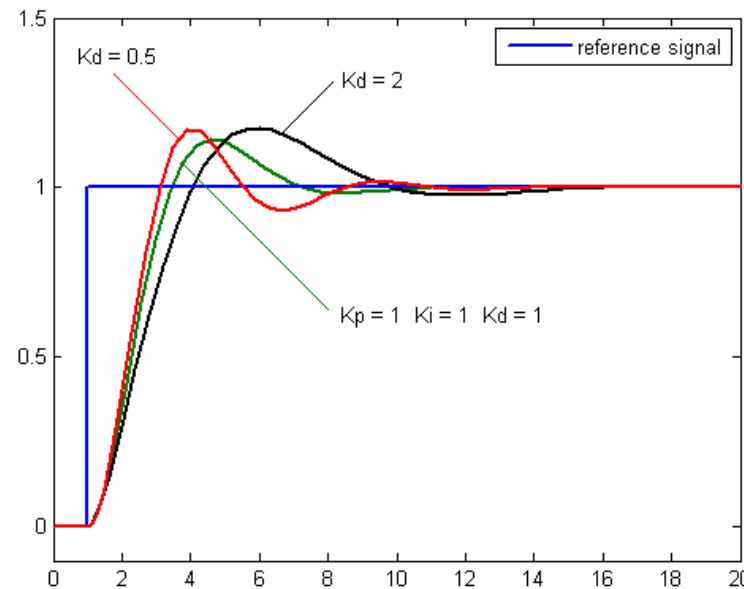
- / accounts for **past values** of the error
 - E.g. if the current output is not sufficiently strong, the integral of the error will accumulate over time
→ the controller will respond by applying a stronger action



Derivative Term

$$K_d \frac{de(t)}{dt}$$

- D accounts for possible future trends of the error, based on its current rate of change
 - E.g. Although large positive control output brings the error closer to zero, it may also produce a large negative error in the near future → the derivative turns negative and the D module reduces the strength of the action to prevent this overshoot



PID can be **PI**, **PD** or just **P**!

- Depending on the system you are trying to control, not all constant terms are essential

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

- By setting the less useful constant to **zero**, you can easily get:
 - ❑ **P** controller, i.e. $K_i = K_d = 0$
 - ❑ **PI** controller, i.e. $K_d = 0$
 - ❑ **PD** controller, i.e. $K_i = 0$
- Keep this in mind when you try it out!

PID Controller is **Continuous**

- The PID controller is continuous:
 - ❑ Adjustment is constantly updated based on real time readings of the plant output and the set point
- So, need to **discretized** the controller for computer implementation
 - ❑ i.e. Plant output is **sampled** at certain time interval
 - ❑ Controller output is similarly calculated at interval

Sample Discretized PID Implementation

error_prior = 0

integral = 0

*K_P = Some **magic** value*

*K_I = Some **magic** value*

*K_D = Some **magic** value*

while(1) {

error = desired_value – actual_value

*integral = integral + (error*iteration_time)*

derivative = (error – error_prior)/iteration_time

*output = K_P*error + K_I*integral + K_D*derivative*

error_prior = error

sleep(iteration_time)

}

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Sampling Rate

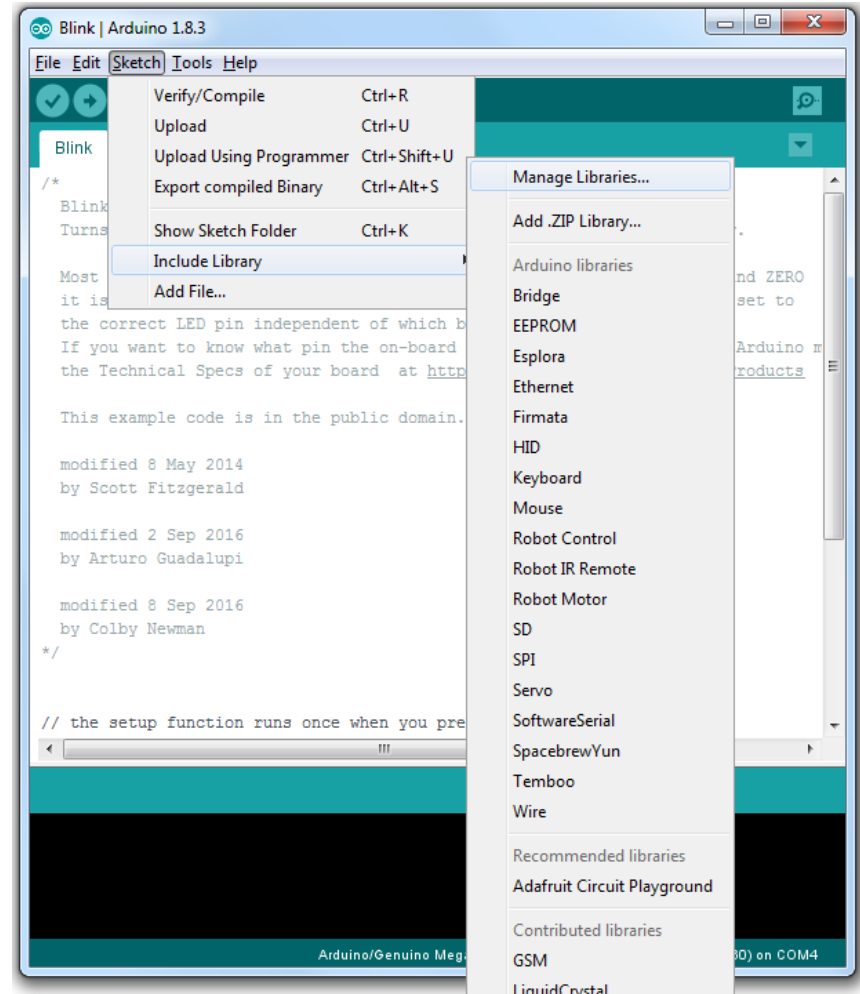
- Pick a good sampling time: $1/10$ to $1/100$ of **settling time**
 - Should be relatively precise, within 1% – use a timer interrupt
 - **Not too fast** → variance in delta t
 - **Not too slow** → too much lag time
- Sampling time changes relative effect of P, I and D

PID on Arduino

- There is a PID library on Arduino!

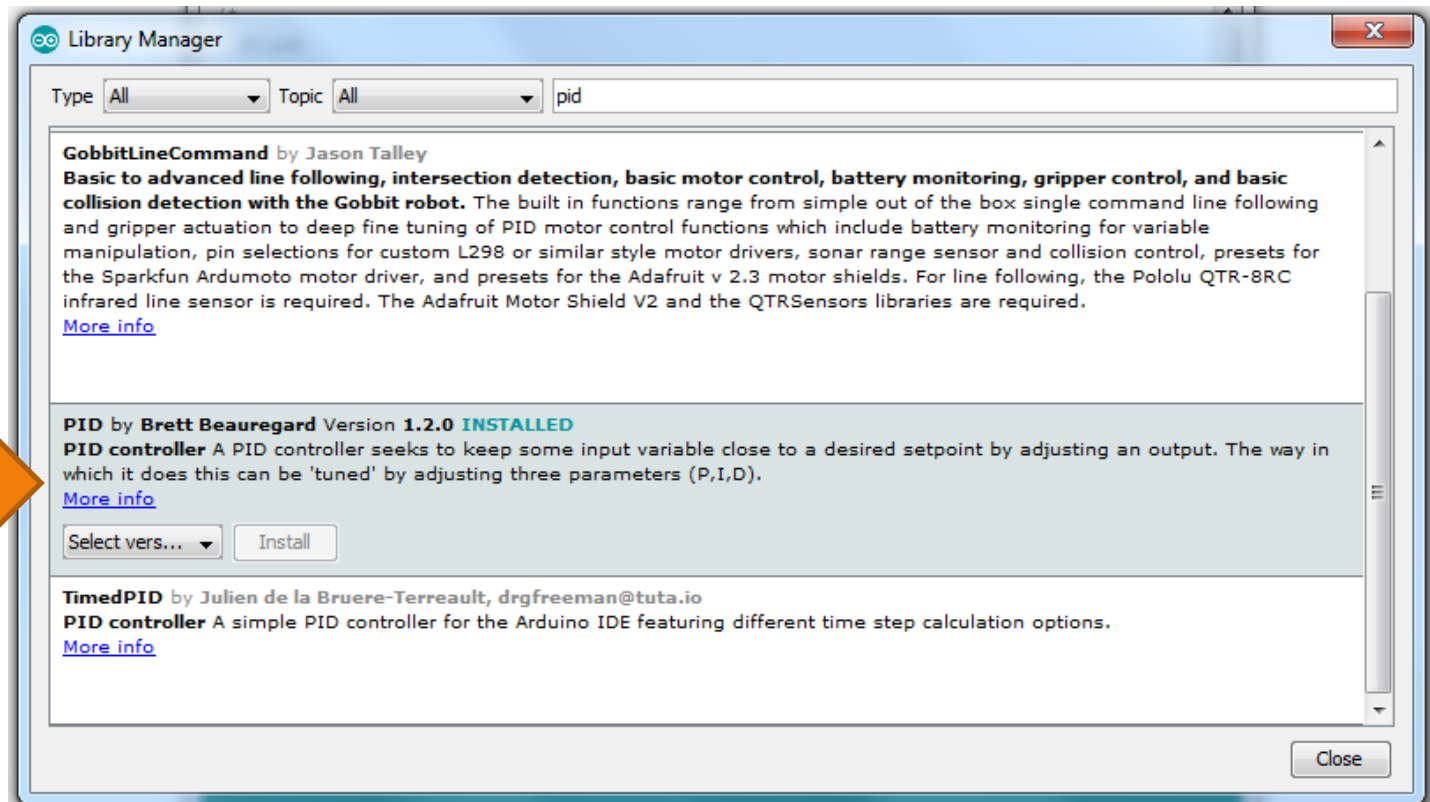
- Step 1:

- ❑ Click "Sketch
→ Include library
→ Manage Libraries"



PID on Arduino

2. Search for "PID", locate the library by *Brett Beauregard*
3. Click "Install"



PID on Arduino: Basics

- Use the "PID_Basic" example
 - Under "File→Examples→PID→..."
- Key Functions:
 - **PID** (&Input, &Output, &Setpoint, Kp, Ki, Kd, Direction)
 - **Compute**()
- Find out more at :
 - <https://playground.arduino.cc/Code/PIDLibrary>

Tuning PID Controller?

Effects of *increasing* a parameter independently

Parameter	Rise time	Overshoot	Settling time	Steady-state error	Stability
K_p	Decrease	Increase	Small change	Decrease	Degrade
K_i	Decrease	Increase	Increase	Eliminate	Degrade
K_d	Minor change	Decrease	Decrease	No effect in theory	Improve if K_d small

- There are a large number of tuning approaches:
 - ❑ Manual (with heuristic, no math)
 - ❑ Manual (with math based heuristics)
 - ❑ Software
 - ❑ etc

Tuning: Heuristic

1. Set K_d and K_i to 0
2. Increase K_p until oscillation, then decrease by factor of 2-4
3. Increase K_i until loss of stability, then back off
4. Increase K_d to increase performance in response to disturbance
5. Iterate from step 2

Ideas for Your Advanced Model Project?

1. What can we control via PID Control?
2. What is the set point?
3. What is the control input?

How much can we help?

- **NO**: we cannot give you a good set of K_p , K_i and K_d value
 - i.e. tuning the PID is your responsibility
- **NO**: tell you exactly where PID can be used
- **YES**: we can help to discuss whether your usage of PID make sense
- **YES**: Help on syntax of the PID library

References

- PID Control, Verification, and Scheduling
 - by David Kohanbash
 - <http://robotsforroboticists.com/pid-control/>
- PID Control Lecture Notes
 - University of Washington
- Control Theory Lecture Note
 - by Colin Tan